

Screening Assignment

Submitted by: Bokka Sowjanya

B.E CSE (AI & ML) | Sai Vidya Institute of Technology, Bengaluru

sowjanya9020@gmail.com | +91-9113050716

Section 1: LLM Project

Project: Emergency Detection Research Q&A Bot (RAG Pipeline)

Problem It Solved

During my major project on AI-based emergency detection, I had over 40 research papers, conference documents, and technical reports. Every time my team needed to find a specific accuracy metric or a referenced technique, we spent 20+ minutes searching manually. Basic keyword search missed semantic context, and general AI tools hallucinated facts not in our papers.

So I built a RAG-based Q&A bot that lets anyone ask natural language questions about my published research paper and get accurate, cited answers — instantly.

High-Level Architecture

The system follows a complete RAG pipeline:

- PDF Ingestion: Research paper loaded using PyPDF2, text extracted page-by-page
- Chunking: Text split into 800-word chunks with 100-word overlap to preserve context
- Vector Embedding: ChromaDB automatically embeds each chunk using all-MiniLM-L6-v2
- Storage: All embeddings stored in ChromaDB vector database
- Query Processing: User question embedded using same model, top-3 similar chunks retrieved
- Generation: Llama 3.3 70B (via Groq API) answers using only retrieved context with page citations
- UI: Gradio-based ChatGPT-style interface with quick question buttons

Engineering Challenge

The hardest part was the chunking strategy. Initially I used `chunk_size=100` with `overlap=500` — which caused crashes because overlap was larger than the chunk itself. I fixed this to `chunk_size=800` with `overlap=100` after understanding that overlap must always be smaller than the chunk size, and that 800 words preserves enough context for technical content without flooding the LLM context window.

Another challenge: Gemini and HuggingFace APIs had quota/compatibility issues. I switched to Groq API with Llama 3.3 70B which gave faster, more reliable results for free.

Links

GitHub Repository :- <https://github.com/bokka-sowjanya/emergency-detection-rag-bot>

Live Demo :- <https://19cf922b53bc19cc11.gradio.live>

Section 2: Architecture Design

Task: Design a RAG pipeline for a document Q&A system over 10,000 internal documents (PDFs, Word files, Confluence pages) where users ask natural language questions and expect accurate, cited answers.

1. Chunking Strategy

I would use Recursive Character Text Splitter with:

- `chunk_size` = 800 tokens — large enough to preserve context, small enough for precise retrieval
- `chunk_overlap` = 100 tokens — prevents cutting sentences mid-way
- Separators: paragraph breaks first, then sentences, then words

Why not smaller chunks? Technical documents lose context. Why not larger? Retrieval becomes noisy and LLM context fills up.

2. Embedding Model

I would use all-MiniLM-L6-v2 (sentence-transformers) because:

- Free, fast, and runs locally — no API cost for 10,000 documents
- 384-dimensional embeddings — compact but highly accurate for semantic search
- Same model must be used for both document embedding and query embedding — this ensures the vector space is consistent

3. Vector Store

ChromaDB for this scale because:

- Handles 10,000+ documents efficiently with fast similarity search
- No SQL needed — vector DB stores embeddings and retrieves by similarity, not keywords
- Built-in HNSW indexing — approximate nearest neighbor search is much faster than brute force

Why not SQL? SQL uses exact keyword matching. Vector DB uses cosine similarity — it finds semantically similar content even if exact words don't match.

4. Retrieval Strategy

- Primary: Semantic similarity search (cosine similarity) — top-k=4 chunks
- Enhanced: Hybrid search = semantic + keyword BM25 — catches both semantic and exact matches
- Reranking: Cross-encoder reranker to re-score top retrieved chunks for precision

Top-k=4 is optimal — too few misses context, too many (k=50) floods the LLM and reduces answer quality.

5. Handling Partial Documents

When a document only partially answers the question:

- Return all retrieved chunks with their source document and page number
- Prompt the LLM to explicitly state what is known and what is missing
- Add a confidence indicator — 'Answer is partial, based on pages 3 and 7 only'

Biggest Failure Mode & Mitigation

Failure Mode: Retrieval misses — the right chunk is in the database but not retrieved because the query is phrased differently than the document.

Mitigation:

- Use hybrid search (semantic + BM25) to catch both semantic and keyword matches
 - Query expansion — rephrase the question 2-3 ways and retrieve for each
 - Add reranker to re-score chunks after initial retrieval
-

Section 3: Debug Challenge — Find and Fix the Bugs

The following code has 6 bugs. Each bug is identified, explained, and corrected below.

Bug 1 — `chunk_overlap > chunk_size` (CRASH)

Location: `CharacterTextSplitter` initialization

Problem: `overlap (500)` is larger than `chunk_size (100)`. This causes an error or infinite loop because every chunk would overlap more than its own content.

```
# WRONG
splitter = CharacterTextSplitter(chunk_size=100, chunk_overlap=500)
# FIXED
splitter = CharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
```

Bug 2 — Wrong Embedding Model Name (CRASH)

Location: `OpenAIEmbeddings` initialization

Problem: `'gpt-4'` is a chat/completion model, not an embedding model. Using it for embeddings throws an API error.

```
# WRONG
embeddings = OpenAIEmbeddings(model="gpt-4")
# FIXED
embeddings = OpenAIEmbeddings(model="text-embedding-ada-002")
```

Bug 3 — `temperature=1.0` for Factual Q&A (WRONG ANSWERS)

Location: `ChatOpenAI` initialization

Problem: Temperature 1.0 makes the model very creative and random. For factual document Q&A, you need `temperature=0` to get deterministic, accurate answers.

```
# WRONG
llm = ChatOpenAI(model="gpt-4", temperature=1.0)
# FIXED
llm = ChatOpenAI(model="gpt-4", temperature=0)
```

Bug 4 — `k=50` Too Many Chunks (PERFORMANCE + QUALITY)

Location: retriever `search_kwargs`

Problem: Retrieving 50 chunks floods the LLM context window, increases cost dramatically, and degrades answer quality because the model gets confused by too much irrelevant content.

```
# WRONG
retriever=vectorstore.as_retriever(search_kwargs={"k": 50})
# FIXED
retriever=vectorstore.as_retriever(search_kwargs={"k": 4})
```

Bug 5 — `return_source_documents=False` (MISSING CITATIONS)

Location: RetrievalQA.from_chain_type

Problem: Source documents are disabled so users cannot see which document or page the answer came from. The assignment requires cited answers.

```
# WRONG
return_source_documents=False
# FIXED
return_source_documents=True
```

Bug 6 — No Error Handling (PRODUCTION CRASH)

Location: Query execution

Problem: Any API failure, network issue, or empty result will crash the entire application silently with no useful error message.

```
# WRONG
result = qa_chain.run("What is our refund policy?")
print(result)
# FIXED
try:
    result = qa_chain.run("What is our refund policy?")
    print(result)
except Exception as e:
    print(f"Error during QA chain execution: {e}")
```